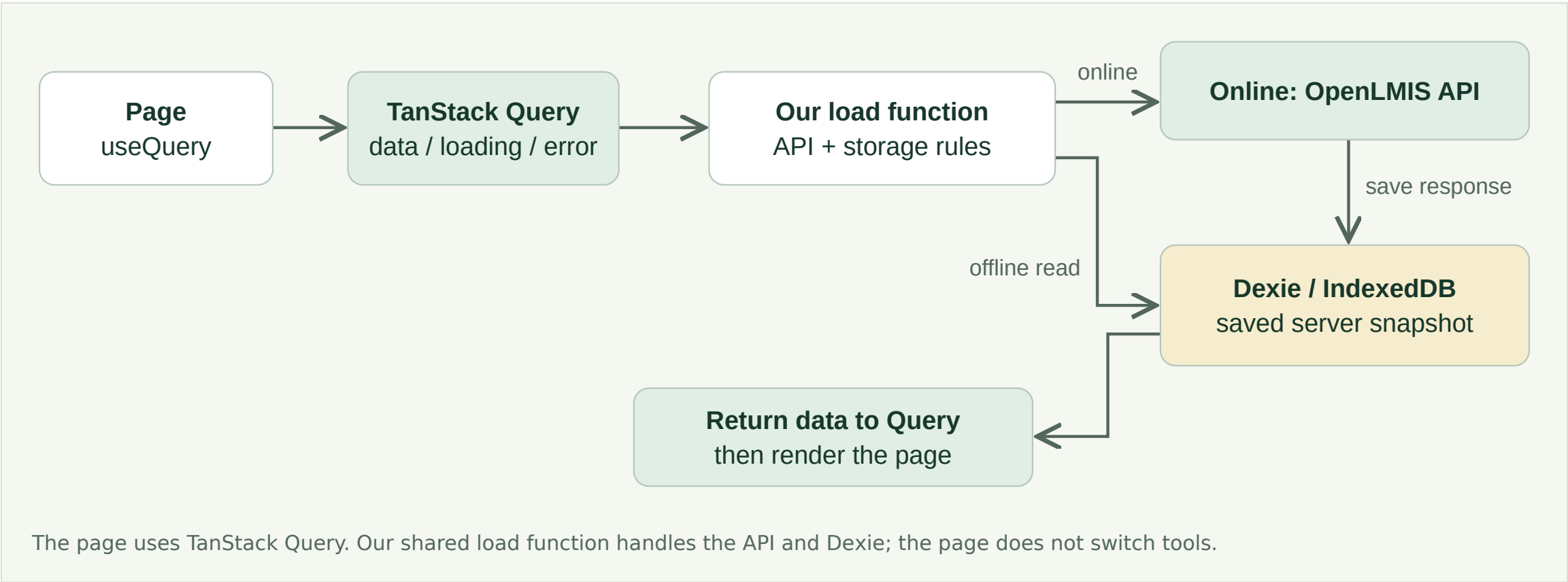


# Keep the stack. Add local storage.

Build one small online workflow first. Then make its data and drafts available offline, using the same component-facing query hooks.

| TOOL                      | WHAT IT DOES                                                                          |
|---------------------------|---------------------------------------------------------------------------------------|
| TanStack Query            | Data for components, loading/error handling, memory caching and refetching.           |
| Dexie / IndexedDB         | Durable downloaded data, editable drafts and pending changes on the device.           |
| Zustand                   | Session and shared UI state. No extra copy of requisitions or other business records. |
| Axios                     | Calls the existing OpenLMIS APIs.                                                     |
| vite-plugin-pwa / Workbox | Caches application files so the UI can open or refresh offline.                       |

## How a page reads data



**Online:** fetch through Axios, store a server snapshot in Dexie, and return data to Query. Download the dependencies required by the workflow, such as its template and relevant orderables.

**Offline:** already displayed data stays visible. After a refresh or browser restart, the app opens from its asset cache and the load function reads saved data from Dexie.

If required data was never downloaded, show “Connect to download this data.” Configure local reads to execute offline and finish with data, an unavailable result or a handled error, rather than leaving Suspense or a loader waiting indefinitely.

## A small shared integration

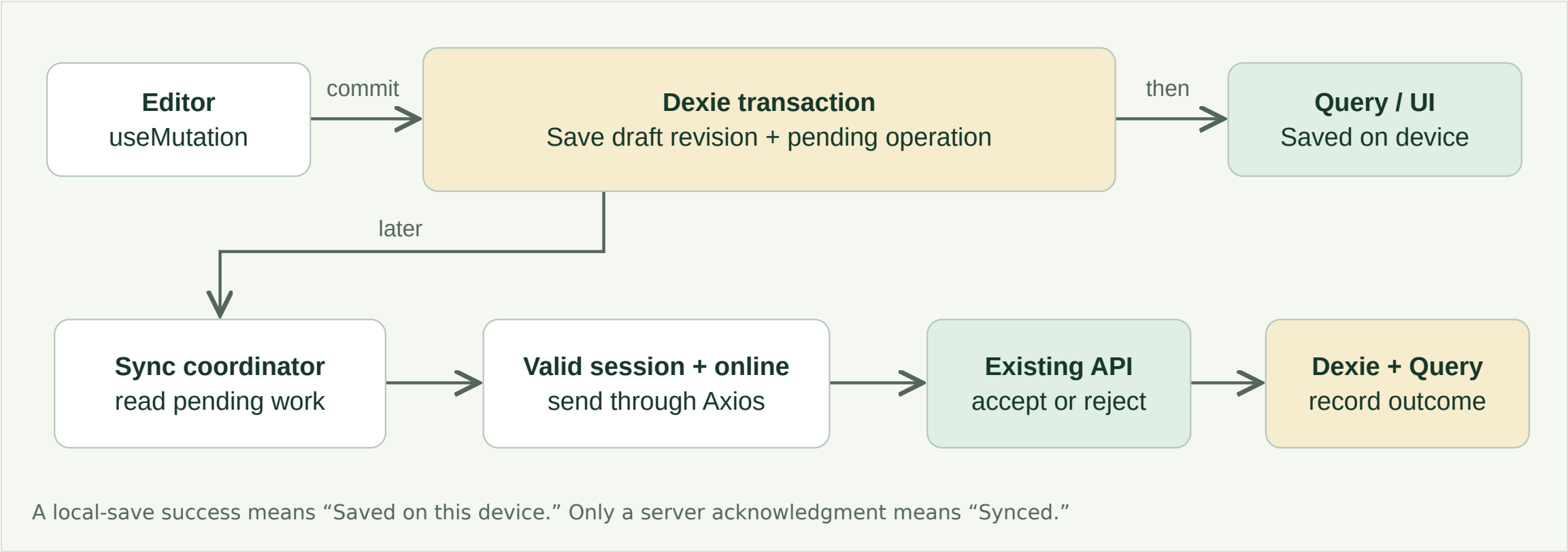
Query and Dexie do not synchronize automatically. Our data-access helpers save/read records and update the relevant Query entries when local data changes. Establish that pattern in the first workflow before generalizing it.

**One durable store.** Query is a disposable memory cache; Dexie stores the data that must survive a restart. We do not also persist the whole Query cache initially.

Cache application assets only in the service worker, not API or OAuth responses. An initial online visit and completed data preparation are required before a workflow is available offline.

# Save locally. Sync when possible.

Keep the first implementation narrow: editing and saving an existing draft. Submission, authorization and approval can follow separately.



## What we save in Dexie

Keep downloaded server snapshots, editable local drafts and pending operations separate. Scope them by deployment and user, retaining each document’s facility/program context. A server refresh must not overwrite an edited draft. Commit local saves before confirming success. When needed, save the draft and its pending operation in one transaction. Record revisions so an acknowledgment of an older upload cannot mark newer edits as synced.

## What happens with auth and reconnecting

The first login requires connectivity. A previously signed-in user can continue permitted local work using cached rights. Token expiry pauses uploads and asks for reauthentication; it does not delete drafts or eject the user from the editor. Refine the existing session state and guards, capture `expires_in`, preserve the login request’s Basic header, and ignore failures from older sessions. Explicit logout still ends local access; warn before deliberately discarding unsynced work. After reconnecting and validating authentication, send eligible pending changes through the existing APIs. Record acknowledgments in Dexie and update Query. Explain rejected or conflicting changes while preserving the draft. A lost response may mean the server already applied the write, so retry only when the endpoint makes that safe.

## Implementation order

- Fix auth handling and inspect the chosen API.** Establish the local-access policy and verify save, retry and conflict behavior. Check the size of data needed for one workflow.
- Build a working online draft slice.** Open, edit and save an existing draft against real endpoints, with templates and validation. Use shared data-access functions. This is a testable baseline, not the entire requisition feature.
- Add offline storage and shell caching.** Persist required data and drafts in Dexie. Prove the app and committed draft reopen offline.
- Add synchronization.** Test reconnecting, expired credentials, newer local edits and an interrupted upload. Use Playwright for real-browser behavior and existing Vitest tests for individual rules.

**Essential rules:** no silent deletion of committed drafts; no cross-user mixing; no blind retry of an uncertain write. Browser storage is not a backup against device loss or cleared site data.

**Consider later:** additional workflows, broader rollout controls, advanced diagnostics, background sync and encrypted multi-user offline access. Implement basic failure messages and recovery as part of the first workflow.

**References:** [Query network modes](#) · [Dexie transactions](#) · [Vite PWA](#) . Dexie, vite-plugin-pwa and Workbox are free open-source libraries; no paid service is required.